

Cinema Seat Reservation System

Piotr Franc and Piotr Szymiec

1 Project Description

The task was to build a cinema seat reservation system using Apache Cassandra as the database. Users can pick a movie, choose seats, and make a reservation, with the system ensuring two people can't book the same seat at the same time. To prevent double-booking we used Cassandra's lightweight transactions - conditional updates that only apply if a given condition holds, which guarantees that reservations are atomic even under concurrent access.

The project is split into a database connection layer, a service layer with all the reservation logic, and a frontend made with tkinter. The GUI shows a seat grid with three modes: reserve, update, and cancel. Three movies are pre-loaded at startup with 40 seats each (rows A-D, columns 1-10).

To test how the system holds up under load we wrote five stress tests covered in the task description.

2 Database Schema

We kept the schema simple - just one table in the `cinema` keyspace. The keyspace uses a replication factor of 2, suitable for a local setup.

Column	Purpose
<code>movie_id</code>	Partition key
<code>seat_id</code>	Id of the seat
<code>reserved</code>	Whether the seat is taken
<code>reserved_by</code>	Name of the person who booked it
<code>reservation_time</code>	When the booking was made

Using `movie_id` as the partition key means fetching all seats for a movie is a single partition read, which makes sense since fetching the movie is the most common query.

3 Problems Encountered

3.1 Timeout errors under heavy load

The most noticeable issue was that under heavy load - particularly in the stress tests - Cassandra started throwing timeout errors. This happened because the conditional updates we rely on are relatively expensive, requiring extra coordination between replicas, so flooding the database with them quickly builds up a backlog.

That said, even when timeouts occurred the behaviour was still correct - no seat ever ended up double-booked. The system caught the errors and treated a failed request as a lost race rather than a crash.

3.2 Slow startup initialisation

The seat initialisation on startup inserts all 120 seats one by one with an existence check on each, which turns out to be noticeably slow. For this project it's acceptable, but in a real world system there should be separate scripts for creating the database and running the gui.